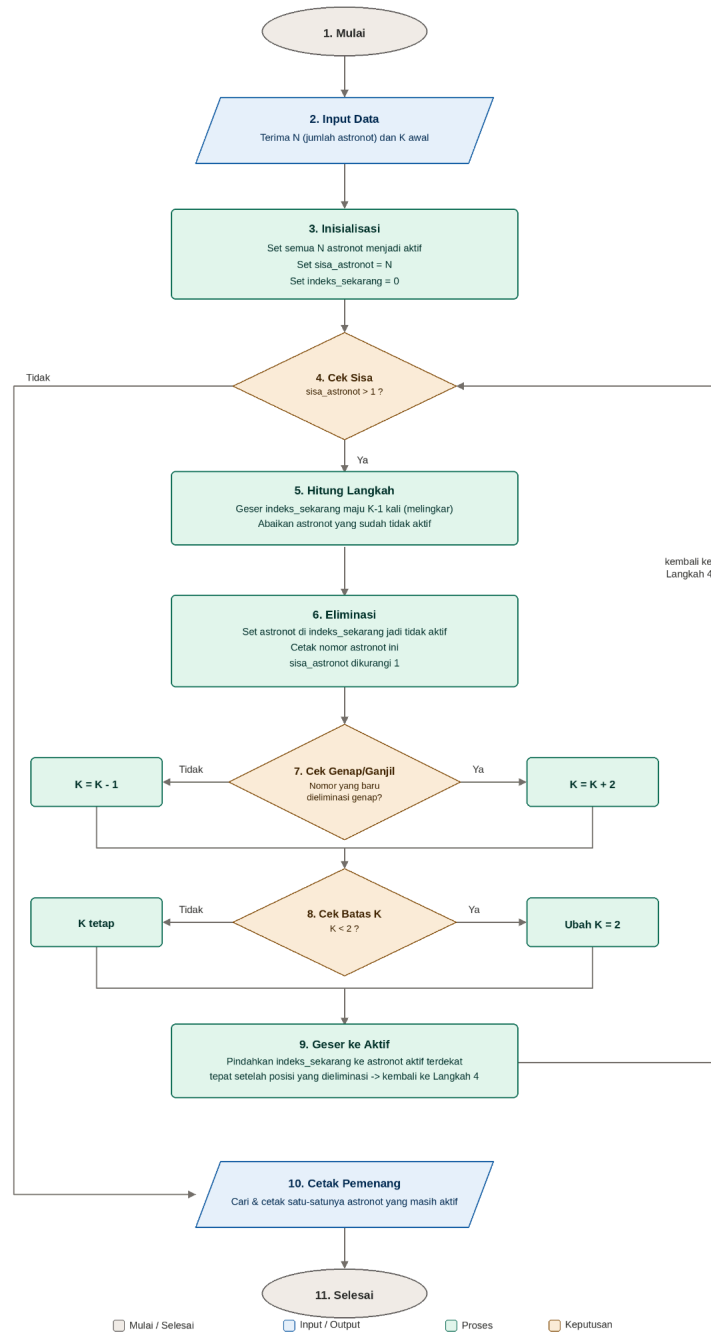


PEMROGRAMAN DASAR TLS

Problem 1: The Last Astronaut



Logik utama penyelesaian masalah ini berdasarkan konsep tatasusunan (array) membulat di mana setiap angkasan yang masih bertahan akan terus berada di dalam gelung penyingkiran. Bermula dengan menerima jumlah keseluruhan angkasan (N) dan nilai kiraan awal (K),

sistem menetapkan status semua angkasawan kepada "aktif". Selagi jumlah angkasawan yang berbaki lebih daripada satu ('sisa_astronot > 1'), program akan mengira sebanyak $K - 1$ langkah ke hadapan secara membulat dari posisi semasa. Pergerakan ini secara automatik mengabaikan mana-mana angkasawan yang telah pun disingkirkan sebelum ini atau berstatus tidak aktif. Angkasawan yang berada pada titik hentian langkah ini akan diubah statusnya kepada "tidak aktif", lalu nombornya dicetak dan direkodkan sebagai angkasawan yang tersingkir, seterusnya mengurangkan baki angkasawan sebanyak satu.

Selepas setiap proses penyingkiran, sistem akan melakukan kemas kini dinamik terhadap nilai K bergantung kepada nombor angkasawan yang baru sahaja disingkirkan. Sekiranya angkasawan tersebut bernombor genap, nilai K akan ditambah sebanyak dua ($K = K + 2$), manakala jika nombor tersebut adalah ganjil, nilai K akan dikurangkan sebanyak satu ($K = K - 1$). Untuk memastikan kiraan kekal rasional, satu syarat kawalan diletakkan di mana jika nilai K jatuh di bawah dua ($K < 2$) selepas perubahan, nilai tersebut akan ditetapkan semula kepada dua. Kedudukan carian untuk pusingan seterusnya kemudian dialihkan kepada angkasawan terdekat yang masih aktif, tepat selepas posisi yang baru dikosongkan tadi. Kitaran ini akan terus berulang sehinggalah hanya tinggal seorang angkasawan, dan program akan mencari lalu mencetak nombor satu-satunya angkasawan yang berstatus aktif tersebut sebagai pemenang mutlak.

Problem 2: Alien-In-The-Middle

DEKLARASI

```
pesan_asli, pesan_sandi : STRING  
panjang, i, nilai_sebelumnya, nilai_sekarang, huruf_baru : INTEGER
```

ALGORITMA

```
READ pesan_asli  
panjang = LENGTH(pesan_asli)  
pesan_sandi = ""  
IF panjang > 0 THEN  
    pesan_sandi = pesan_sandi + pesan_asli[0]  
    nilai_sebelumnya = KONVERSI_ALFABET_KE_ANGKA(pesan_asli[0])  
    FOR i = 1 TO panjang - 1 DO  
        nilai_sekarang = KONVERSI_ALFABET_KE_ANGKA(pesan_asli[i])  
        huruf_baru = nilai_sekarang + nilai_sebelumnya  
        IF huruf_baru > 26 THEN  
            huruf_baru = huruf_baru - 26  
        END IF  
        pesan_sandi = pesan_sandi +  
KONVERSI_ANGKA_KE_ALFABET(huruf_baru)  
        nilai_sebelumnya = nilai_sekarang  
    END FOR  
END IF  
PRINT pesan_sandi  
END PROGRAM
```

Logika algoritma ini berpusat pada pergeseran karakter alfabet secara berurutan menggunakan nilai numerik dari karakter aslinya. Proses dimulai dengan membaca teks pesan asli dan langsung menetapkan huruf pertama ke dalam pesan sandi tanpa perubahan apa pun. Huruf pertama tersebut kemudian dikonversi menjadi representasi angka berdasarkan posisinya dalam alfabet, yaitu A bernilai 1 hingga Z bernilai 26. Angka hasil konversi ini disimpan ke dalam variabel `nilai\_sebelumnya` yang akan berfungsi sebagai penentu jumlah pergeseran (shift) untuk huruf kedua.

Untuk huruf kedua dan seterusnya, program menjalankan perulangan dengan mengonversi huruf asli yang sedang diproses menjadi angka, lalu menjumlahkannya dengan `nilai\_sebelumnya` untuk mendapatkan posisi huruf yang baru. Apabila hasil penjumlahan tersebut melebihi batas angka 26, program akan mengurangnya dengan 26 agar perhitungan berputar kembali ke awal alfabet secara sirkular. Setelah posisi huruf baru didapatkan, angka tersebut dikonversi kembali menjadi karakter abjad dan ditambahkan ke hasil akhir pesan sandi.

Sebelum berpindah ke karakter berikutnya, program memperbarui 'nilai_sebelumnya' menggunakan nilai alfabet dari huruf asli yang baru saja diproses, sehingga memastikan bahwa setiap pergeseran huruf selalu bergantung murni pada huruf asli yang berada tepat di posisi sebelumnya.